

Day 7 — ES6 Part 1 Student Tasks

Topics: "use strict", Hoisting, var / let / const, Arrow Functions, Template Literals, Destructuring, Default / Rest / Spread, Memory & Copying

Write your answers in a JS file. For predict-output questions, write the output **and** a short reason.

Part A — "use strict"

1. Without strict mode, create a function that assigns a value to a variable **without** declaring it (var / let / const). Call the function, then print that variable outside the function. What happened?
2. Rewrite the previous function with "use strict" inside it. Use try/catch and print the error message.
3. Inside a strict-mode function, declare var localVar = "test", then try delete localVar. Catch the error and print it. After that, create an object with one property and successfully delete that property.

Part B — Hoisting

4. Predict the output, then run the code:

```
console.log(x);  
var x = 10;  
console.log(x);
```

5. Predict what happens in each case, then explain the difference:

```
// Case 1  
sayHi();  
function sayHi() {  
  console.log("Hi");  
}  
  
// Case 2  
sayBye();  
var sayBye = function () {  
  console.log("Bye");  
};
```

6. Predict the output of this code and explain using the TDZ idea:

```
console.log(a);  
let a = 5;
```

7. Predict the output:

```
var n = 1;
function demo() {
  console.log(n);
  var n = 2;
  console.log(n);
}
demo();
console.log(n);
```

Part C — var vs let vs const

8. Write one example that proves `var` is **function-scoped** (use an `if` or a `for` loop inside a function).
9. Write one example that proves `let` / `const` are **block-scoped**.
10. Show the difference between re-declaration with `var` and with `let` (comment the line that causes an error).
11. Create a `const` object named `student` with `name`, `age`, and `city`. Then:
 - change `age`
 - add a new property `grade`
 - delete `city`
 - try to reassign the whole object (`student = { ... }`) and catch the error

Print the object after each successful change.

12. Create a `const` array `nums = [1, 2, 3]`. Use `push`, change index `0`, then try to reassign the array. What is allowed and what is not?
13. Which of these is valid? Test and write the result for each:

```
var a;
let b;
const c;
```

14. In the browser console, create: `var g1 = "var global"`, `let g2 = "let global"`, `const g3 = "const global"`. Check `window.g1`, `window.g2`, and `window.g3`. Write what you see.
15. Fix this bug by changing only `var` to `let`, then explain why it works:

```
const handlers = {};
for (var i = 0; i < 3; i++) {
  handlers["fn" + i] = function () {
    return "index: " + i;
  };
}
```

```
console.log(handlers.fn0());
console.log(handlers.fn2());
```

Part D — Arrow Functions & Template Literals

16. Convert this function to an arrow function that returns a string in one line:

```
function welcome(name) {
  return "Welcome, " + name + "!";
}
```

Use a template literal in the arrow version.

17. Create an arrow function `fullInfo(first, last, age)` that returns: `"Ali Hassan is 25 years old"` using template literals (with your own values).
18. Write an arrow function `multiply(a, b)` that returns the product. Then write another arrow function with a block body that prints both numbers and returns their sum.

Part E — Destructuring, Default, Rest, Spread

19. Using object destructuring, extract `title`, `price`, and `inStock` from:

```
const product = {
  title: "Laptop",
  price: 15000,
  inStock: true,
  brand: "Dell"
};
```

Print the three extracted variables.

20. Using array destructuring on `["HTML", "CSS", "JS", "React"]`, store the first two items in variables and print them.
21. Create a function `greet(name = "Guest", message = "Hello")` that returns `"Hello, Guest!"` style text. Call it three times: with both args, with only name, and with no args.
22. Write a function `sumAll(...numbers)` that returns the sum of any count of numbers. Test it with `sumAll(1, 2, 3)` and `sumAll(10, 20, 30, 40)`.
23. Using spread:
- merge `[1, 2]` and `[3, 4, 5]` into one array
 - make a copy of `[10, 20, 30]`, push `40` into the copy only, then print both arrays
24. Merge these two objects with spread, then print the result:

```
const user = { name: "Sara", age: 22 };
const contact = { email: "sara@nti.com", age: 23 };
```

Which `age` wins? Why?

25. Pass array values as function arguments using spread:

```
const values = [2, 4, 6, 8];
function total(a, b, c, d) {
  return a + b + c + d;
}
```

Call `total` using spread and print the result.

Part F — Memory & Copying

26. Predict and explain:

```
let person1 = { name: "Ali", child: { age: 5 } };
let person2 = person1;
person2.name = "Omar";
console.log(person1.name);
```

27. Create a shallow copy of this object using spread:

```
const original = {
  name: "Mona",
  details: { city: "Cairo" }
};
```

Change `name` on the copy, then change `details.city` on the copy. Print both objects and explain what changed in the original and why.

28. Using the same `original` object, create a deep copy with `structuredClone` (or `JSON.parse(JSON.stringify(...))`). Change the nested `city` on the deep copy and prove the original did not change.

29. Save this object to `localStorage` under the key `"userdata"`:

```
{ name: "Ahmed", age: 26, city: "Alex" }
```

Then read it back, parse it, print its type and its values, and finally remove the key.

Part G — Challenge Mix

30. Build a small `APP_CONFIG` using `const` with: `name`, `version`, and nested `api: { baseUrl, timeout }`. Change `timeout`, add a feature string into a `features` array, and try reassigning `APP_CONFIG` (catch the error).

31. Create a function `createCard(title, price = 0, ...tags)` that returns an object using:

- default parameter for `price`
- rest for `tags`

- template literal for a `label` property like `"Laptop - 15000 EGP"`

Then create two cards and print them.

32. You have:

```
const students = [  
  { name: "Omar", grade: 80 },  
  { name: "Mona", grade: 90 },  
  { name: "Ali", grade: 70 }  
];
```

Using destructuring inside a loop (or `map`), print each student as: `"Omar scored 80"`.